

COURSE CODE/TITLE:	CPE 212 - Automating Server Management Server	SECTION:	CPE31S2
DATE PERFORMED:	09/07/2026	DATE SUBMITTED:	09/07/2026
NAME:	Javillonar, Madhel kathlyn Ann A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 2.1 Hands-on Activity - SSH Key-Based Authentication and GIT Setup

1. PROCEDURE OUTPUT

Task 1: Create an SSH Key Pair for User Authentication

1. The simplest way to generate a key pair is to run *ssh-keygen* without arguments. In this case, it will prompt for the file in which to store keys. First, the tool asked where to save the file. SSH keys for user authentication are usually stored in the users *.ssh* directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case *id_rsa* when using the default RSA algorithm. It could also be, for example, *id_dsa* or *id_ecdsa*.
2. Issue the command *ssh-keygen -t rsa -b 4096*. The algorithm is selected using the *-t* option and key size using the *-b* option.
3. When asked for a passphrase, just press enter. The passphrase is used for encrypting the key, so that it cannot be used even if someone obtains the private key file. The passphrase should be cryptographically strong.
4. Verify that you have created the key by issuing the command *ls -la .ssh*. The command should show the *.ssh* directory containing a pair of keys. For example, *id_rsa.pub* and *id_rsa*.

```
wdjavillonar@workstation:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/wdjavillonar/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/wdjavillonar/.ssh/id_rsa
Your public key has been saved in /home/wdjavillonar/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:EQq+VFWsjF8LJcJPWntLAjM6ewxIpg+NExRmqf+QDVY wdjavillonar@workstation
The key's randomart image is:
+---[RSA 4096]-----+
|  .o+...*+.      |
| o.OE+oo+++ o    |
| .,.= =0.= +     |
| . o= .o=* .o     |
| o +o ..S+ .     |
| + .   . .       |
|  o              |
|  .              |
|                 |
+-----[SHA256]-----+
wdjavillonar@workstation:~$
```

```
wdjavillonar@workstation:~$ ls -la .ssh
total 24
drwx----- 2 wdjavillonar wdjavillonar 4096 Jul 22 14:17 .
drwxr-x--- 15 wdjavillonar wdjavillonar 4096 Jul 22 10:31 ..
-rw----- 1 wdjavillonar wdjavillonar  0 Jul 22 10:20 authorized_keys
-rw----- 1 wdjavillonar wdjavillonar 3389 Jul 22 14:17 id_rsa
-rw-r--r-- 1 wdjavillonar wdjavillonar  750 Jul 22 14:17 id_rsa.pub
-rw----- 1 wdjavillonar wdjavillonar 1956 Jul 22 13:52 known_hosts
-rw----- 1 wdjavillonar wdjavillonar 1120 Jul 22 13:51 known_hosts.old
wdjavillonar@workstation:~$
```

Task 2: Copying the Public Key to the remote servers

1. To use public key authentication, the public key must be copied to a server and installed in an *authorized_keys* file. This can be conveniently done using the *ssh-copy-id* tool.
2. Issue the command similar to this: *ssh-copy-id -i ~/.ssh/id_rsa user@host*
3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.
4. On the local machine, verify that you can SSH with Server 1 and Server 2. What did you notice? Did the connection ask for a password? If not, why?

SCREENSHOTS:

```
wdjavillonar@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub serv1javillonar@192.168.56.101
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/wdjavillonar/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are
already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to i
nstall the new keys
serv1javillonar@192.168.56.101's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'serv1javillonar@192.168.56.101'"
and check to make sure that only the key(s) you wanted were added.

wdjavillonar@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa.pub serv2javillonar@192.168.56.102
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/wdjavillonar/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are
already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to i
nstall the new keys
serv2javillonar@192.168.56.102's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'serv2javillonar@192.168.56.102'"
and check to make sure that only the key(s) you wanted were added.

wdjavillonar@workstation:~$
```

```
wdjavillonar@workstation:~$ ssh serv1javillonar@192.168.56.101
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-28-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Wed Jul 22 13:55:59 2026 from 192.168.56.103
serv1javillonar@serv1javillonar-VirtualBox:~$ exit
logout
Connection to 192.168.56.101 closed.
wdjavillonar@workstation:~$ ssh serv2javillonar@192.168.56.102
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 7.0.0-28-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
```

Installation of git

```
wdjavillonar@workstation:~$ which git
wdjavillonar@workstation:~$ sudo apt install git
[sudo] password for wdjavillonar:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  git-man liberror-perl
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb git-cvs
  git-mediawiki git-svn
The following NEW packages will be installed:
  git git-man liberror-perl
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 4,806 kB of archives.
After this operation, 24.5 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://ph.archive.ubuntu.com/ubuntu noble/main amd64 liberror-perl all 0.17029-2 [25.6 kB]
Get:2 http://ph.archive.ubuntu.com/ubuntu noble-updates/main amd64 git-man all 1:2.43.0-1ubuntu7
3 [1,100 kB]
14% [2 git-man 389 kB/1,100 kB 35%]

wdjavillonar@workstation:~$ git --version
git version 2.43.0
wdjavillonar@workstation:~$
```

Part 2: Discussion

Provide screenshots for each task. (CHECK BELOW PO THANK YOU)

It is assumed that you are done with the last activity (**Activity 2: SSH Key-Based Authentication**).

Set up Git

At the heart of GitHub is an open-source version control system (VCS) called Git. Git is responsible for everything GitHub-related that happens locally on your computer. To use Git on the command line, you'll need to download, install, and configure Git on your computer. You can also install GitHub CLI to use GitHub from the command line. If you don't need to work with files locally, GitHub lets you complete many Git-related actions directly in the browser, including:

- Creating a repository
- Forking a repository
- Managing files



COMPUTER
ENGINEERING

QUEZON CITY

· Being social

Task 3: Set up the Git Repository

1. On the local machine, verify the version of your git using the command *which git*. If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: *sudo apt install git*
2. After the installation, issue the command *which git* again. The directory of git is usually installed in this location: *user/bin/git*.
3. The version of git installed in your device is the latest. Try issuing the command *git --version* to know the version installed.
4. Using the browser in the local machine, go to www.github.com.
5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub account.
 - a. Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.
 - b. Create a new SSH key on GitHub. Go your profile's setting and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create a new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.
 - c. On the local machine's terminal, issue the command *cat .ssh/id_rsa.pub* and copy the public key. Paste it on the GitHub key and press Add SSH key.
 - d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.
 - e. Issue the command *git clone* followed by the copied link. For example, *git clone git@github.com:jvtaylar-cpe/CPE232_yourname.git*. When prompted to continue connecting, type yes and press enter.
 - f. To verify that you have cloned the GitHub repository, issue the command *ls*. Observe that you have the CPE232_yourname in the list of your directories. Use CD command to go to that directory and LS command to see the file README.md.
 - g. Use the following commands to personalize your git.
 - *git config --global user.name "Your Name"*
 - *git config --global user.email yourname@email.com*
 - Verify that you have personalized the config file using the command *cat ~/.gitconfig*
 - h. Edit the README.md file using nano command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.
 - i. Use the *git status* command to display the state of the working directory and the staging area. This command shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?

- j. Use the command `git add README.md` to add the file into the staging area.
- k. Use the `git commit -m "your message"` to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next commit.
- l. Use the command `git push <remote><branch>` to upload the local repository content to GitHub repository. Pushing means to transfer commits from the local repository to the remote repository. As an example, you may issue `git push origin main`.
- m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice the how long was the last commit. It should be some minutes ago and the message you typed on the git commit command should be there. Also, the README.md file should have been edited according to the text you wrote.

SCREENSHOTS

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#)
Required fields are marked with an asterisk (*).

1

General

Owner *



MadhelKathlynAnnJavillonar

Repository name *

CPE232_Javillonar

✓ CPE232_Javillonar is available.

Great repository names are short and memorable. How about [glowing-funicular?](#)

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

MadhelKathlynAnnJavillonar / CPE232_Javillonar

Search Type to search

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

CPE232_Javillonar (Public)

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file Code

MadhelKathlynAnnJavillonar Initial commit 04a7243 · now 1 Commit

README.md Initial commit now

README

CPE232_Javillonar

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published Create a new release

Packages

No packages published Publish your first package

Contributors 1

MadhelKathlynAnnJavillonar

SSH keys

[New SSH key](#)

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys



CPE232 Key

SHA256: EQq+VFwSjF8LJcJPWnt1AkJM6ewxIpg+NExRmqf+QDVY

Added on Jul 22, 2026

Never used — Read/write

Delete

Check out our guide to [connecting to GitHub using SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

[New GPG key](#)

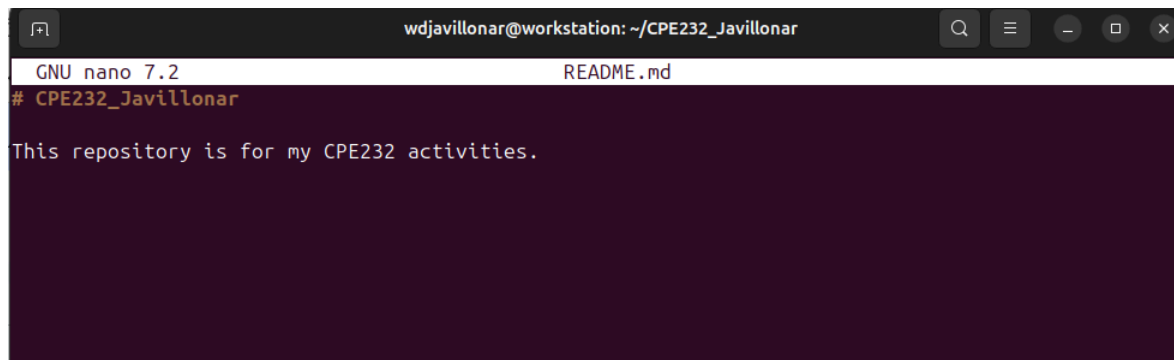
There are no GPG keys associated with your account.

Learn how to [generate a GPG key and add it to your account](#).

```
wdjavillonar@workstation:~$ git clone git@github.com:MadhelKathlynAnnJavillonar/CPE232_Javillonar
.git
Cloning into 'CPE232_Javillonar'...
The authenticity of host 'github.com (20.205.243.166)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
wdjavillonar@workstation:~$
```

```
wdjavillonar@workstation:~$ ls
CPE232_Javillonar  Documents  Music      Public  Templates
Desktop            Downloads  Pictures   snap    Videos
wdjavillonar@workstation:~$ cd CPE232_Javillonar
wdjavillonar@workstation:~/CPE232_Javillonar$ ls
README.md
wdjavillonar@workstation:~/CPE232_Javillonar$
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ git config --global user.name "Madhel Kathlyn Javillonar"
wdjavillonar@workstation:~/CPE232_Javillonar$ git config --global user.email "qmkaajavillonar@tip.edu.ph"
wdjavillonar@workstation:~/CPE232_Javillonar$ cat ~/.gitconfig
[user]
    name = Madhel Kathlyn Javillonar
    email = qmkaajavillonar@tip.edu.ph
wdjavillonar@workstation:~/CPE232_Javillonar$ nano README.md
```



The screenshot shows a terminal window with a nano editor running. The title bar indicates the file is README.md. The editor content shows the first line of the README file, which states the repository is for CPE232 activities.

```
GNU nano 7.2 README.md
# CPE232_Javillonar

This repository is for my CPE232 activities.
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

wdjavillonar@workstation:~/CPE232_Javillonar$ git add README.md
wdjavillonar@workstation:~/CPE232_Javillonar$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

wdjavillonar@workstation:~/CPE232_Javillonar$ git commit -m "Update README"
[main 09f8987] Update README
 1 file changed, 3 insertions(+), 1 deletion(-)
wdjavillonar@workstation:~/CPE232_Javillonar$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 324 bytes | 324.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:MadhelKathlynAnnJavillonar/CPE232_Javillonar.git
   04a7243..09f8987  main -> main
wdjavillonar@workstation:~/CPE232_Javillonar$
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflections:

Answer the following:

1. What sort of things have we so far done to the remote servers using ansible commands?
So far, we have used Ansible to manage and perform tasks on the remote servers without having to manually work on each server one by one. It allows us to send commands and carry out tasks from one machine, making server management easier and more organized.
2. How important is the inventory file?

The inventory file is important because it tells Ansible which servers it needs to manage. It contains

the information about the remote machines, so Ansible knows where to connect and which servers the commands should be sent to. Without a proper inventory file, it would be difficult for Ansible to identify and manage the correct servers.

Conclusions/Learnings:

In this activity, I learned more about managing remote servers using SSH, Ansible, and Git. I also learned how SSH keys can make connecting to servers easier without entering the password every time. I struggled a lot while doing the activity because I made some mistakes with typos and encountered errors, especially when mounting and setting up the virtual machines. The virtual machines also lagged sometimes, which made the process take longer. Even though I had difficulties, I was able to troubleshoot the errors and continue with the activity. Overall, this activity helped me become more familiar with server management and taught me to be more careful when typing commands and checking errors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Chatgpt and Quillbot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: I used ChatGPT to summarize and clarify the activity instructions. I used QuillBot for grammar correction.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used AI only to help me understand the activity and improve my grammar. I still did the actual activity and completed the required tasks myself.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.